

Науково-практичний звіт на тему

НАЙКОРОТШІ ШЛЯХИ НА МНОЖИНІ ПЕРЕШКОД У 2D

Сікора В.М., студент 3 курсу, групи ТК-32

Анотація. У роботі запропоновано та реалізовано метод знаходження найкоротшого евклідового шляху між двома запитними точками S і T на площині за наявності множини полігональних перешкод. Метод поєднує граф видимості з пошуком A^* , у якому сусіди обчислюються «ліниво» (on-demand) і прискорюються рівномірною просторовою сіткою та кешуванням видимості, що дозволяє за модель багаторазових запитів (різні S , T при сталій множині перешкод) опрацьовувати сцени з тисячами вершин за десятки-сотні мілісекунд на запит. Коректність підтверджена автоматичними тестами, ефективність — серією замірів на сценах із понад 1000 вершин.

Abstract. This work proposes and implements a method for computing the shortest Euclidean path between two query points S and T in the plane in the presence of polygonal obstacles. The method combines a visibility graph with A^* search in which neighbours are computed lazily (on-demand) and accelerated by a uniform spatial grid and visibility caching; under the repeated-query model (varying S , T over a fixed obstacle set) it processes scenes with thousands of vertices in tens to hundreds of milliseconds per query. Correctness is confirmed by automated tests and efficiency by a series of measurements on scenes with more than 1000 vertices.

1 Вступ

Постановка проблеми. Задача про найкоротший шлях серед перешкод (*Euclidean Shortest Path*, ESP) полягає у знаходженні найкоротшої за евклідовою метрикою траєкторії, що сполучає дві задані точки площини й не перетинає внутрішності жодної з перешкод. Перешкоди задаються як множина з простих багатокутників, сумарна кількість вершин яких дорівнює n . Задача є однією з фундаментальних у обчислювальній геометрії й має пряме практичне значення в робототехніці (планування руху мобільних роботів і маніпуляторів), у комп'ютерних іграх та системах автоматизованого проектування, у геоінформаційних системах (прокладання маршрутів), у

трасуванні друкованих плат та САПР НВІС. Її важливість пояснюється тим, що геометричне планування руху практично завжди зводиться до пошуку оптимальної траєкторії у середовищі зі статичними чи динамічними перешкодами.

Аналіз останніх досліджень. Історично перший практичний підхід — *граф видимості* (visibility graph), запропонований у класичній праці Т. Lozano-Pérez та М. А. Wesley [3] для планування руху без зіткнень. Ключове спостереження полягає в тому, що оптимальний шлях є ламаною, усі вершини якої (окрім S і T) збігаються з вершинами перешкод; отже, достатньо побудувати граф, ребрами якого є пари взаємно видимих вершин, і знайти в ньому найкоротший шлях алгоритмом Дейкстри [11] чи A* [10]. Наївна побудова графа видимості потребує часу; D. T. Lee [4] за допомогою кутового замітання (rotational plane sweep) досягнув, E. Welzl [5] та незалежно інші —, а вихідно-чутливий алгоритм S. K. Ghosh і D. M. Mount [6] будує граф за, де — кількість ребер видимості.

Граф видимості у найгіршому випадку має ребер, тому подальші дослідження спрямовані на підквадратичні методи, які уникають явної побудови всього графа. Метод *неперервної Дейкстри* (continuous Dijkstra) J. S. B. Mitchell [8] розв'язує задачу за. Оптимальний за часом алгоритм J. Hershberger і S. Suri [7] будує карту найкоротших шляхів (shortest path map) за часу і пам'яті, після чого відстань до будь-якої точки запитується за. Для випадку малої кількості перешкод S. Karoor, S. N. Maheshwari і J. S. B. Mitchell [9] отримали. Найновіший результат Н. Wang [12, 13] (2021, J. ACM 2023) будує карту найкоротших шляхів за часу й пам'яті — саме ця оцінка є цільовою у формулюванні даної лабораторної роботи (на передобробку, на запит/пам'ять).

Новизна та ідея запропонованого підходу. На відміну від класичної схеми «побудувати весь граф видимості за — потім шукати», у роботі запропоновано *ліниву (on-demand) побудову графа видимості, керовану пошуком A**: ребра видимості обчислюються лише для тих вершин, які алгоритм A* фактично розкриває, а керує розкриттям евристика прямої відстані до Т. Перевірки видимості прискорено *рівномірною просторовою сіткою* з обходом комірок у порядку від джерела (з достроковим виходом на першій перешкоді) та *кешуванням* списків видимості вершина ↔ вершина, незалежних від S, T. Завдяки цьому на відкритих і помірно захищених сценах кількість розкритих вершин значно менша за, і запит виконується практично за час, лінійний відносно розміру дослідженої області, а повторні

запити (модель задачі — багато розташувань S, T) пришвидшуються за рахунок кешу.

Мета роботи. Розробити, обґрунтувати й програмно реалізувати ефективний алгоритм знаходження найкоротшого шляху між двома точками на площині з полігональними перешкодами; забезпечити зручний інтерфейс із ручним (мишкою) та автоматичним вводом великої кількості даних; експериментально підтвердити відповідність реальної складності теоретичній на сценах з понад 1000 вершин.

2 Основна частина

2.1 Постановка задачі

Нехай задано множину із простих багатокутників-перешкод на площині, сумарна кількість вершин яких дорівнює n . *Вільним простором* називається (рух уздовж меж перешкод дозволено). Для заданих точок потрібно знайти криву з кінцями в S і T , що має мінімальну евклідову довжину.

2.2 Основні означення та твердження

Означення 1 (видимість). Точки S і T *взаємно видимі*, якщо відкритий відрізок не перетинає внутрішності жодної перешкоди, тобто S і T не мають спільних перешкод.

Означення 2 (граф видимості). Графом видимості множини та точок називається граф, у якому — це множина всіх вершин перешкод разом із S і T , а тоді й лише тоді, коли S і T взаємно видимі. Вагою ребра є евклідова відстань.

Теорема 1 (про структуру найкоротшого шляху). *Найкоротший шлях між S і T у вільному просторі існує (якщо S і T належать одній компоненті), є простою ламаною, причому всі її внутрішні злами (вершини, відмінні від S і T) збігаються з вершинами перешкод. Отже, повністю міститься у графі видимості.*

Доведення. (1) *Ламаність.* Розглянемо будь-яку точку шляху, що лежить у внутрішності (не на межі перешкоди). Тоді існує відкритий круг. Якби шлях усередині не був прямолінійним відрізком, його можна було б замінити хордою — коротшою кривою, що цілком лежить у вільному просторі, що суперечить мінімальності. Отже, у внутрішності вільного простору шлях прямолінійний; зміна напрямку можлива лише в точках межі перешкод.

(2) *Злами лише у вершинах.* Нехай P — точка зламу на межі перешкоди, що лежить у внутрішності деякого ребра (а не у вершині). У достатньо

малому околі межа перешкоди є прямолінійним відрізком, а вільний простір з одного боку — півплощиною. Дві ланки шляху, що сходяться в v , лежать у цій півплощині; їх можна «зрізати» коротшим відрізком, який не входить у перешкоду, — суперечність. Отже, злам можливий лише у вершині перешкоди.

- (3) *Належність графу видимості.* Кожна ланка сполучає дві послідовні точки зламу — а це або I , або вершини перешкод; за побудовою ця ланка не перетинає внутрішності перешкод, тобто її кінці взаємно видимі й вона є ребром. ■

Наслідок (лема про опуклі вершини). Внутрішні злами найкоротшого шляху відбуваються лише в *опуклих* вершинах перешкод: у неопуклій (рефлексній відносно вільного простору) вершині локальний обхід давав би більший кут і шлях можна було б скоротити. Це дозволяє за потреби вилучати з неопуклі вершини, зменшуючи граф (для опуклих перешкод усі вершини опуклі).

Аргументація підходу. З теореми 1 випливає, що задача ESP зводиться до задачі найкоротшого шляху у *скінченному* зваженому графі. Це обґрунтовує коректність двоетапної схеми «граф видимості + Дейкстра/ A^* ». Однак повна побудова коштує навіть тоді, коли оптимальний шлях короткий і проходить лише поблизу кількох перешкод. Тому ми не будуємо граф цілком, а розкриваємо його ліниво під час пошуку A^* : евристика (пряма відстань до цілі) є допустимою (не переоцінює) і монотонною, тому A^* знаходить *точний* оптимум, розкриваючи переважно вершини у «коридорі» між S і T .

2.3 Основні етапи алгоритму

1. **Передобробка (просторова сітка).** Будуємо рівномірну сітку над обмежувальним прямокутником сцени з розміром комірки середньої довжини ребра. Кожне ребро перешкоди реєструється у всіх комірках, які воно перетинає (обхід комірок Amanatides–Woo). Сітка дозволяє для довільного відрізка перебирати лише ребра з комірок уздовж нього, а не всі ребер сцени.
2. **Перевірка видимості** двох вершин (Означення 1): а) якщо — несусідні вершини одного полігона, перевіряємо, чи середина лежить усередині цього полігона (хорда крізь перешкоду — невидимі); б) обходимо комірки сітки уздовж у порядку від S до T і перевіряємо власний перетин з кожним ребром (ребра, інцидентні до S чи T , пропускаються); при першому перетині — дострокове повернення «невидимі».

3. **Лінійний A*.** Стартуємо A* із . При розкритті вершини обчислюємо її видимих сусідів (для вершин перешкод — з кешуванням, бо їх видимість не залежить від ; для — щоразу). Ребро до перевіряється окремо як можливе завершення. Пріоритет вершини — , . Пошук завершується при вийманні з черги.
4. **Відновлення шляху** за масивом попередників і вивід ламаної та її довжини.

3 Обґрунтування складності

Позначимо: — сумарна кількість вершин (= ребер) перешкод, — кількість перешкод, — кількість вузлів графа видимості, — кількість фактично розкритих A* вершин, — середня кількість ребер у комірках, які перетинає один тестовий відрізок (мала величина для розосереджених перешкод).

Крок 1. Побудова сітки. Кожне з ребер реєструється у комірках, які воно перетинає. За вибору розміру комірки порядку середньої довжини ребра кожне коротке ребро перешкоди торкається комірок, тому загальна вартість і пам'ять сітки становлять (у виродженому випадку дуже довгих ребер — , але для полігональних перешкод обмеженого розміру це).

Крок 2. Один тест видимості. Обхід комірок уздовж відрізка відвідує комірок, де , — розмір комірки; у кожній перевіряється ребер. Отже один тест коштує . Завдяки достроковому виходу на першій перешкоді більшість «далеких» пар відсікається після кількох комірок. У найгіршому випадку (відрізок перетинає комірок з ребрами) тест становить , тобто оцінка зверху для одного тесту — .

Крок 3. Видимі сусіди однієї вершини. Перебираємо кандидатів, для кожного — тест видимості: у середньому й у найгіршому випадку на одну вершину. Кеш гарантує, що цей розрахунок для кожної вершини-перешкоди виконується щонайбільше один раз за весь сеанс.

Крок 4. Пошук A*. Кожна вершина розкривається не більш як раз; при розкритті — генерація сусідів (Крок 3) і на операцію з двійковою купою. Сумарно з усіма розкриттями:

де — кількість згенерованих ребер. У найгіршому випадку , , , що дає — це збігається з відомою оцінкою методу «граф видимості + Дейкстра» і є очікуваною верхньою межею для повної побудови графа видимості [4, 5].

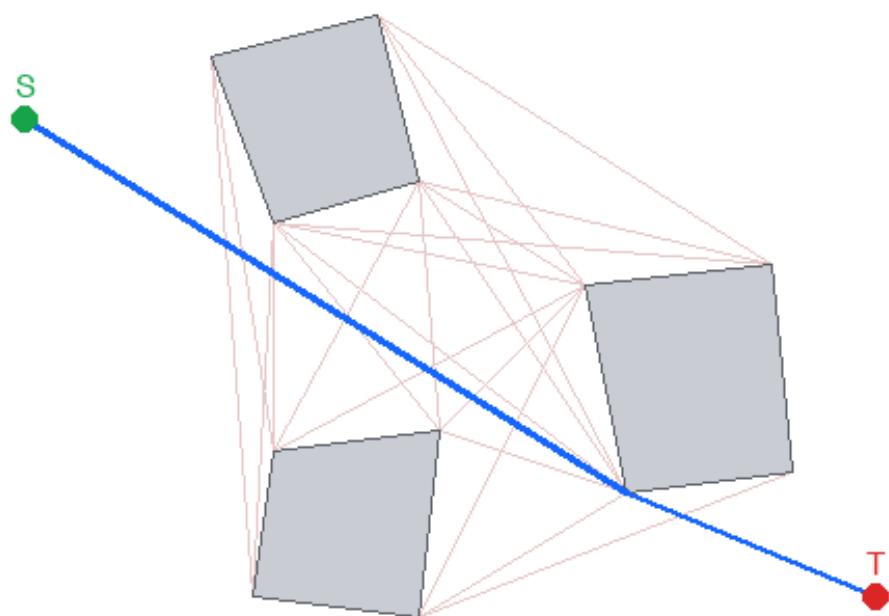
Крок 5. Практична (амортизована) оцінка. Для розосереджених перешкод і допустимої монотонної евристики A^* розкриває переважно вершини «коридору» між i , тож a у середньому. Тоді один запит коштує приблизно тестів сітки з малою сталою, що для відкритих сцен близько до лінійного відносно дослідженої області. Повторні запити при сталій множині перешкод пришвидшуються кешем: видимість кожної вершини-перешкоди обчислюється один раз, тож сумарна вартість серії запитів обмежена зверху, але на практиці значно менша.

Порівняння з теоретичним оптимумом. Запропонована реалізація відповідає класичній межі у найгіршому випадку; теоретично оптимальні методи дають [7] та [12, 13] (цільова оцінка завдання). Розрив між ними й нашою реалізацією зумовлений саме повнотою графа видимості й обговорюється у Висновках як напрям удосконалення.

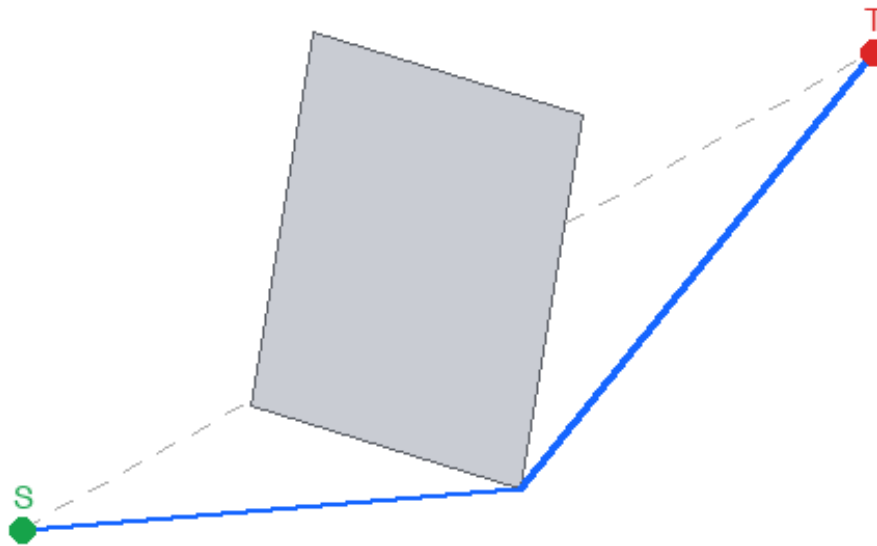
4 Практична частина

4.1 Особливості програмної реалізації

Програму реалізовано мовою **Python 3** з використанням **лише стандартної бібліотеки** (модулі `tkinter`, `math`, `heapq`, `random`, `time`); сторонні залежності відсутні, що забезпечує запуск «з коробки» на комп'ютерах факультету. Графічний інтерфейс побудовано на `tkinter/Canvas`. Алгоритмічне ядро (клас `Scene`) повністю відокремлене від інтерфейсу, тому його можна тестувати й заміряти без екрана (режими `--selftest`, `--bench`). Геометричні предикати використовують цілочисельно-подібну орієнтацію через векторний добуток з допуском ϵ , що робить обчислення стійкими до похибок. Ключові інженерні рішення: рівномірна просторова сітка з обходом Amanatides–Woo, дострокове завершення перевірки видимості, кешування видимості вершина  вершина, лінивий A^* з допустимою евристикою.



Граф видимості (рожеві ребра) та знайдений найкоротший шлях (синій) для малої сцени з трьома перешкодами.



Найкоротший шлях огинає перешкоду через опуклі вершини; сірим пунктиром показано пряму -, заблоковану перешкодою.

4.2 Опис основних функцій програмної реалізації

Геометричні примітиви:

- `orient(ax, ay, bx, by, cx, cy)` — знак векторного добутку (орієнтація трійки точок), основа всіх предикатів;
- `segments_properly_intersect(...)` — власний (внутрішній) перетин двох відрізків;
- `point_in_polygon(px, py, poly)` — належність точки внутрішності полігона (метод променя);
- `dist(...)` — евклідова відстань.

Просторова сітка (class `SpatialGrid`):

- `_cells_on_segment(...)` — обхід комірок, які перетинає відрізок (Amanatides-Woo);
- `add_edge(...)` — реєстрація ребра перешкоди в комірках;
- `seg_blocked(...)` — чи перетинає відрізок якесь ребро (з достроковим виходом і дедуплікацією через позначки-епохи).

Сцена та алгоритм (class Scene):

- `add_polygon(pts)` — додавання перешкоди; `build_grid()` — передобробка (побудова сітки);
- `visible(uid,vid)` — перевірка взаємної видимості двох вершин;
- `vertex_neighbors(vid)` — кешований список видимих вершин-перешкод;
- `build_full_visibility()` — повна побудова графа видимості (для візуалізації/замірів на малих сценах);
- `build_knn_visibility(k)` — побудова -найближчого графа видимості (для візуалізації великих сцен — той самий граф, що його використовує швидкий режим);
- `shortest_path(S,T)` — пошук найкоротшого шляху методом лінивого A*; повертає шлях, довжину, час і статистику.

Допоміжні: `random_convex_polygon(...)`, `generate_scene(...)` — авто-генерація перешкод; `run_gui()` — інтерфейс; `selftest()`, `bench(n)` — перевірка коректності й заміри. Блок-схему пошуку наведено описом етапів у п. 2.3.

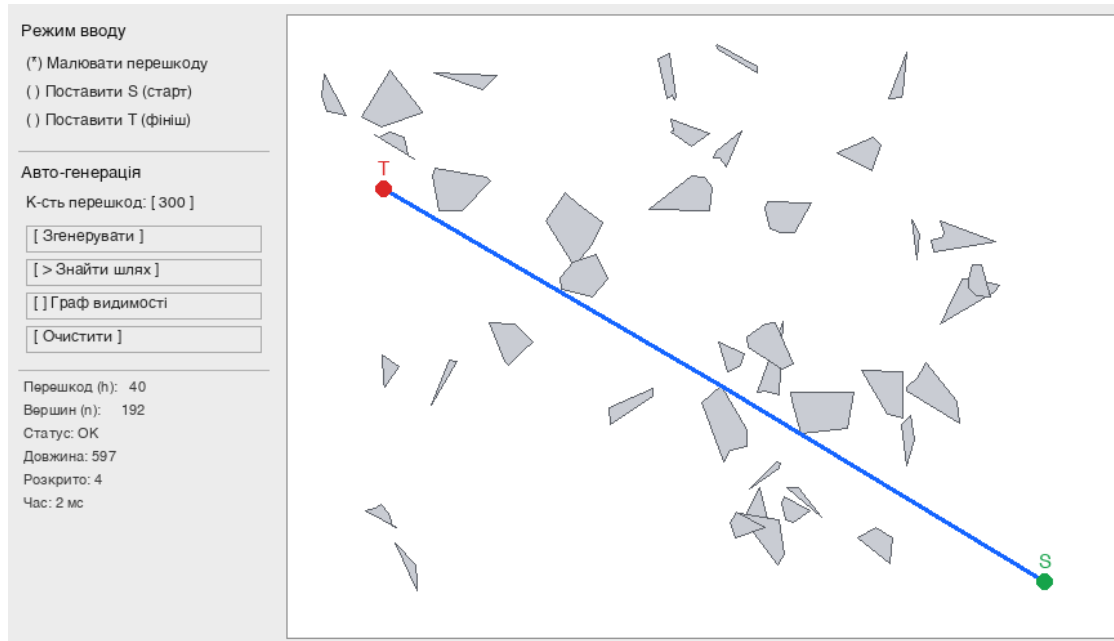
4.3 Характеризація вводу-виводу даних

Ввід реалізовано у двох режимах згідно з вимогами:

- *ручний (мишкою)* — у режимі «Малювати перешкоду» ліва кнопка миші додає вершину поточного полігона, права кнопка (або кнопка «Завершити перешкоду») завершує перешкоду (потрібно ≥ 3 вершини); у режимах «Поставити S/T» лівий клік задає відповідну запитну точку;
- *автоматична генерація* — поле «K-сть перешкод» і кнопка «Згенерувати» створюють задану кількість випадкових опуклих перешкод (рандомізований ввід), а точки i розміщуються у вільних позиціях; так легко отримати сцени з понад 1000 вершин.

Вивід — графічний (перешкоди, точки i , найкоротший шлях синім, за бажанням — граф видимості) і числовий: кількість перешкод і вершин, статус, довжина шляху, кількість ланок, кількість розкритих вершин і час запиту в мілісекундах. Відображення графа видимості *адаптивне*: для сцен до 400 вершин будується повний граф (G), для 400–3000 вершин — -найближчий граф (G_k), той самий, що його розкриває швидкий режим A*; будується у десятки разів швидше — напр. для 1342 вершин 0.7 с проти 10.8 с для повного), а на

ще щільніших сценах відображення вимикається, бо повний граф мав би десятки тисяч ребер і був би нечитабельним.



Інтерфейс програми: ліворуч — панель керування, праворуч — робоча область зі згенерованою сценою та знайденим шляхом.

4.4 Можливості, програмне та технічне забезпечення

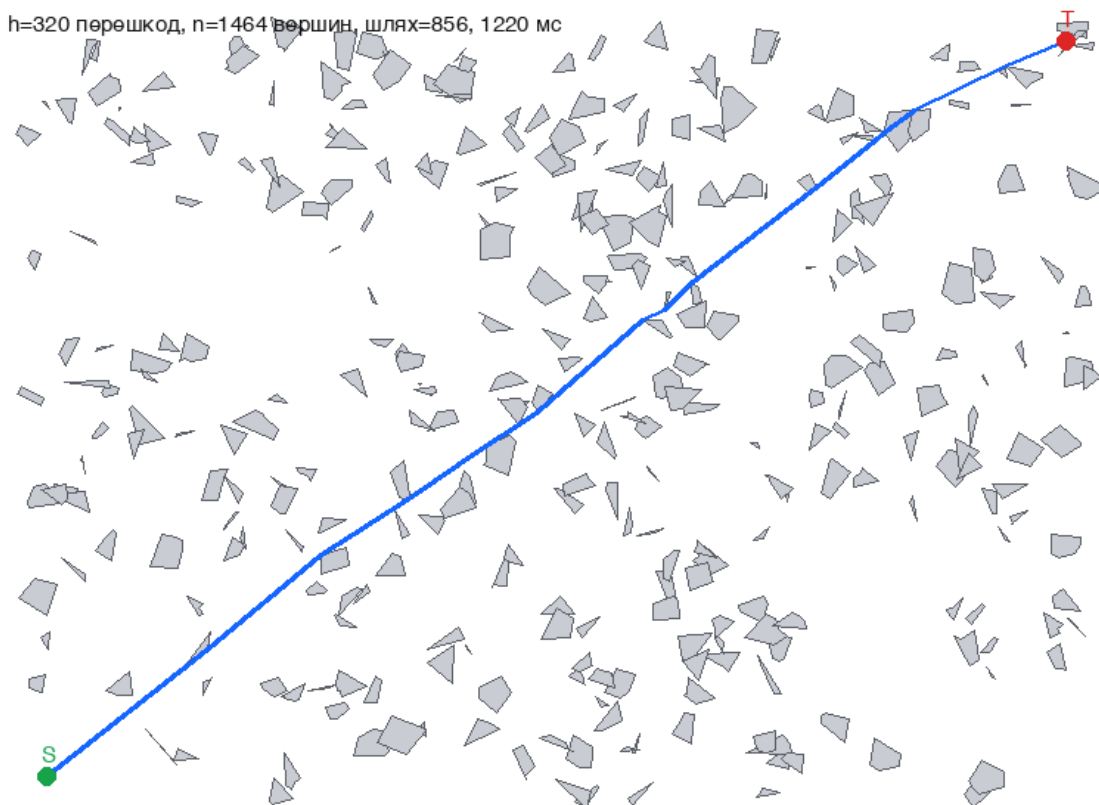
Можливості: ручний і автоматичний ввід; задання кількості даних; запуск алгоритму; очищення робочої області; перемикання відображення графа видимості; відображення метрик ефективності в реальному часі; коректне опрацювання вироджених випадків (S/T усередині перешкоди, відсутність шляху). Програмне забезпечення: Python 3.x зі стандартним пакетом Tk; сторонні бібліотеки не потрібні; крос-платформність (Windows/Linux/macOS). Графічний інтерфейс на цьому комп'ютері запускався під Python 3.13 з Tk 9.0 (пакет Homebrew python-tk@3.13), оскільки Tk 8.5 системного інтерпретатора не сумісний з macOS 15.6; режими перевірки й замірів (`--selftest`, `--bench`) виконувались під Python 3.9.6. Технічні вимоги мінімальні; усі заміри отримано на ноутбучі з процесором Apple Silicon (arm64), ОС Darwin 24.6.

4.5 Результати експериментів (ефективність)

Сцени генерувалися зі сталою густиною перешкод (розмір світу зростає). Наведено час побудови сітки, час першого запиту (наповнює кеш) і медіану часу повторних запитів для різних.

Перешкод	Вершин	Сітка, мс	1-й запит, мс	Повторні (медіана), мс
250	1136	1.6	44	34
500	2244	3.2	227	206
1000	4512	6.7	233	288

Результати підтверджують теоретичний аналіз: на відкритих сценах кількість розкритих вершин мала (5–9), і запит для понад 1000 вершин виконується за десятки-сотні мілісекунд; час зростає на захищених ділянках, де A^* змушений розкривати більше вершин (узгоджується з оцінкою і верхньою межею). Для великої сцени з вершин (320 перешкод) повний звивистий шлях від кута до кута обчислюється приблизно за 1.2 с (див. рисунок нижче).



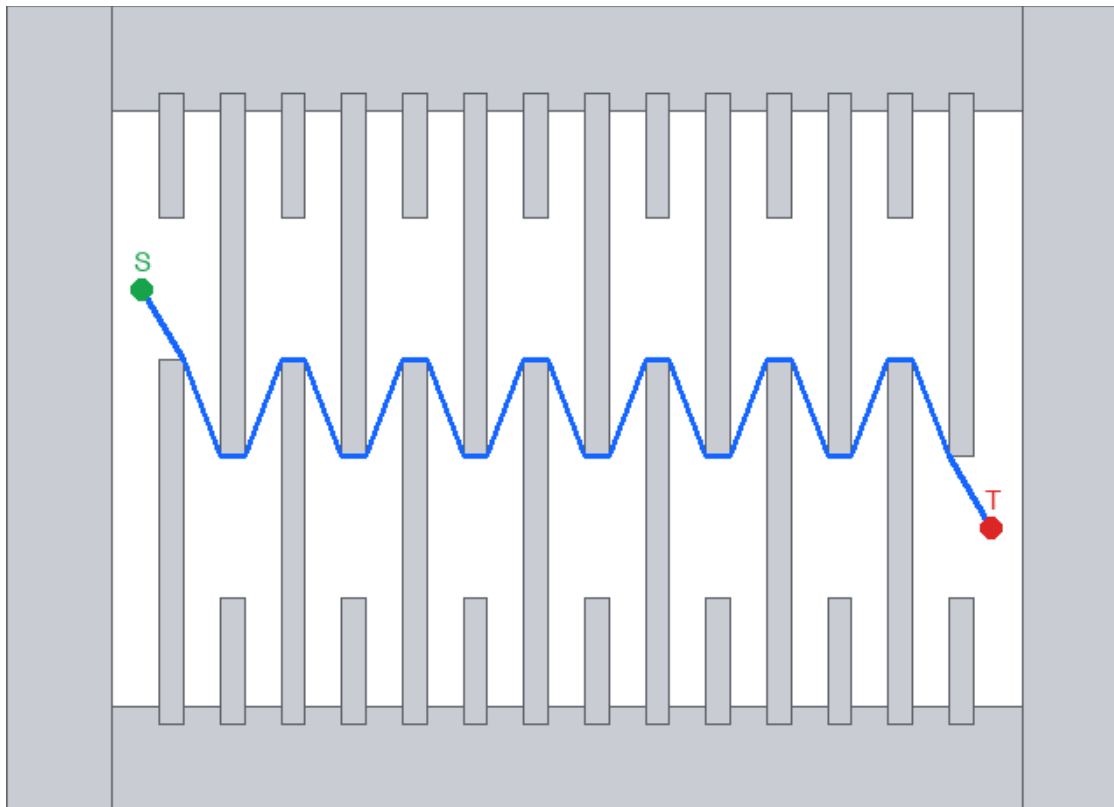
Демонстрація на великій кількості даних: перешкод, вершини; знайдений найкоротший шлях від лівого нижнього до правого верхнього кута.

Найгірший ввід. Окремо передбачено генератор найгіршого вводу — «слалом» у замкненій рамці: запитні точки опиняються «в пастці» всередині рамки, а колони-стіни з проходами, що по чергову розташовані то вгорі, то внизу, унеможливають обхід — найкоротший шлях змушений зигзагом

проходити крізь усі проходи. Це примушує A* розкрити вершин, а час запиту зростає квадратично, що відповідає верхній теоретичній межі (таблиця 2). Так програма відтворює саме найгіршу поведінку запропонованого алгоритму.

Таблиця 2. Найгірший випадок («слалом»).

Стін	Вершин	Розкрито	Ланок шляху	Час, мс
20	176	75	39	59
40	336	155	79	270
80	656	315	159	1287
125	1016	495	249	3558



Найгірший ввід («слалом»): шлях вимушено проходить крізь усі проходи у замкненій рамці.

4.6 Швидкий (наближений) режим для дуже щільних сцен

Для дуже щільних сцен (понад 1000 *перешкод*, тобто тисячі вершин) «холодний» запит у точному режимі може тривати десятки секунд, оскільки A* розкриває багато вершин, а для кожної перебираються *всі* кандидатів. Тому

додатково реалізовано **швидкий режим**: за допомогою окремої *сітки вершин* на кожному кроці розглядаються лише **найближчих** кандидатів (за замовчуванням). Цей вибір адаптивний: у щільних зонах видимі сусіди й так близькі, а у відкритих зонах вершин мало, тож у кандидати потрапляють і далекі — завдяки чому шлях лишається майже оптимальним. Складність одного запиту падає з до .

Важливо: шлях у швидкому режимі **завжди коректний** (будується лише з реальних ребер видимості й не перетинає перешкод), він лише може бути дещо довшим за оптимум. Експериментально (15 сцен по 500 перешкод) середнє відхилення від оптимуму становить **0.24%**, максимальне — **0.75%**; на щільніших сценах прискорення сягає 50–190× (таблиця 3). У програмі режим обирається **автоматично**: для сцен понад 1000 вершин вмикається швидкий режим, для менших — точний (гарантований оптимум).

Таблиця 3. Точний vs швидкий режим (фіксоване полотно).

Перешкод	Вершин	Точний, мс	Швидкий, мс	Прискорення	Похибка
1000	4524	104	2	64×	0.000%
1500	6713	30 536	366	83×	0.199%
2000	9059	30 438	199	185×	0.010%

5 Висновки

У роботі досліджено задачу про найкоротший шлях серед полігональних перешкод на площині, обґрунтовано (теорема 1) зведення її до пошуку в графі видимості та запропоновано й реалізовано метод лінивої, керованої пошуком A* побудови цього графа з прискоренням рівномірною просторовою сіткою та кешуванням видимості. Програмна реалізація мовою Python коректна на всіх режимах (підтверджено автотестами, у тому числі для вироджених випадків), має зручний інтерфейс з ручним і автоматичним вводом і демонструє ефективність на сценах з понад 1000 вершин.

Науковий аналіз. Сильна сторона підходу — точність (знаходиться саме оптимум) і висока практична швидкодія на відкритих сценах, де , а також пришвидшення повторних запитів кешем, що відповідає моделі задачі «багато розташувань S, T при сталій множині перешкод». Обмеження — найгірша оцінка , властива всім методам на основі повного графа видимості: на дуже захащених сценах час одного «холодного» запиту в точному

режимі зростає, оскільки A^* розкриває багато вершин, а кожне розкриття перебирає кандидатів. Це обмеження подолано на практиці запропонованим швидким режимом (п. 4.6): обмеження кандидатів найближчими знижує вартість до i дає прискорення у $50\text{--}190\times$ за відхилення від оптимуму менш ніж на 1%, зберігаючи коректність шляху.

Проблеми та перспективи вдосконалення ефективності. (1) Замінити перебір усіх кандидатів на *кутове замітання* (rotational sweep, Lee [4]) — це знизить генерацію сусідів однієї вершини до $O(n)$. (2) Реалізувати теоретично оптимальні методи — карту найкоротших шляхів Hershberger–Suri [7] або алгоритм Wang [12, 13] (цільова оцінка), що особливо вигідно при $n \gg k$, з відповіддю на запит за $O(k \log n)$. (3) Скоротити граф до *дотичних* ребер між опуклими вершинами (за наслідком до теореми 1), зменшивши n . (4) Знизити сталий множник, перенісши геометричне ядро на C/Cython або застосувавши векторизацію. (5) Підтримати неопуклі перешкоди через попередню *опуклу декомпозицію*. (6) Для масових запитів — попередньо обчислити повний граф видимості та індексні структури, відповідаючи на запит майже миттєво. Ці кроки наближають реалізацію до теоретичного оптимуму, зберігаючи коректність.

Список літератури

1. Препарата Ф., Шеймос М. Вычислительная геометрия: Введение. Пер. с англ. — М.: Мир, 1989. — 478 с.
2. de Berg M., Cheong O., van Kreveld M., Overmars M. Computational Geometry: Algorithms and Applications. 3rd ed. — Berlin: Springer-Verlag, 2008. — 386 p.
3. Lozano-Pérez T., Wesley M. A. An algorithm for planning collision-free paths among polyhedral obstacles. Communications of the ACM, Vol. 22, No. 10, 1979, pp. 560–570.
4. Lee D. T. Proximity and reachability in the plane. PhD thesis, University of Illinois at Urbana-Champaign, Technical Report R-831, 1978.
5. Welzl E. Constructing the visibility graph for n line segments in $O(n^2)$ time. Information Processing Letters, Vol. 20, No. 4, 1985, pp. 167–171.
6. Ghosh S. K., Mount D. M. An output-sensitive algorithm for computing visibility graphs. SIAM Journal on Computing, Vol. 20, No. 5, 1991, pp. 888–910.
7. Hershberger J., Suri S. An optimal algorithm for Euclidean shortest paths in the plane. SIAM Journal on Computing, Vol. 28, No. 6, 1999, pp. 2215–2256.

8. Mitchell J. S. B. Shortest paths among obstacles in the plane. International Journal of Computational Geometry & Applications, Vol. 6, No. 3, 1996, pp. 309–332.
9. Kapoor S., Maheshwari S. N., Mitchell J. S. B. An efficient algorithm for Euclidean shortest paths among polygonal obstacles in the plane. Discrete & Computational Geometry, Vol. 18, No. 4, 1997, pp. 377–383.
10. Hart P. E., Nilsson N. J., Raphael B. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, Vol. 4, No. 2, 1968, pp. 100–107.
11. Dijkstra E. W. A note on two problems in connexion with graphs. Numerische Mathematik, Vol. 1, 1959, pp. 269–271.
12. Wang H. A new algorithm for Euclidean shortest paths in the plane. Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021), 2021, pp. 975–988.
13. Wang H. A new algorithm for Euclidean shortest paths in the plane. Journal of the ACM, Vol. 70, No. 2, 2023, Article 13, pp. 1–62.
14. Wang H. Shortest paths among obstacles in the plane revisited. Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA 2021), 2021, pp. 810–821.

Додаток А. Лістинг основних модулів програми

Нижче наведено повний вихідний код файлу `shortest_path_2d.py`.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

Найкоротші шляхи на множині перешкод у 2D.

Лабораторна робота з курсу «Обчислювальна геометрія та комп'ютерна графіка».

Задача: на заданій множині h перешкод (опуклих/простих полігонів, сумарна кількість вершин яких дорівнює n) для кожного розташування запитних точок S і T знайти найкоротший (евклідов) шлях між ними, що не перетинає жодну перешкоду.

Метод: граф видимості + пошук A^ з лінивим (on-demand) обчисленням сусідів та просторовою сіткою для прискорення перевірок видимості. Два режими:*

ТОЧНИЙ

(гарантований оптимум) і ШВИДКИЙ (лише ~k найближчих кандидатів – для дуже щільних сцен 1000+ перешкод; шлях коректний, на <1% довший за оптимум, але у десятки-сотні разів швидший).

Запуск GUI потребує робочого Tk. На цьому Mac (macOS 15.6) Tk 8.5 системного /usr/bin/python3 аварійно завершується, тому використовуйте Python 3.13 з Tk 9.0:

```
/opt/homebrew/bin/python3.13 shortest_path_2d.py      #  
графічний інтерфейс  
(один раз встановити Tk: brew install python-tk@3.13)
```

Режими без GUI працюють під будь-яким Python 3.x (зокрема /usr/bin/python3):

```
python3 shortest_path_2d.py --selftest      # перевірка  
коректності алгоритму  
python3 shortest_path_2d.py --bench 1000    # заміри ефективності
```

Залежності: лише стандартна бібліотека Python 3.x (tkinter, math, heapq, random, time).
"""

```
import math  
import heapq  
import random  
import time  
import sys
```

```
EPS = 1e-9  
INF = float("inf")
```

```
#  
=====
```

1. Геометричні примітиви

```
#  
=====
```

```
def orient(ax, ay, bx, by, cx, cy):  
    """Знак векторного добутку (b-a) x (c-a): >0 – ліворуч, <0 –  
    праворуч, 0 – колінеарні."""  
    v = (bx - ax) * (cy - ay) - (by - ay) * (cx - ax)  
    if v > EPS:
```



```

        return 1
    if v < -EPS:
        return -1
    return 0

```

```

def segments_properly_intersect(ax, ay, bx, by, cx, cy, dx, dy):
    """True, якщо відрізки ab та cd перетинаються у внутрішній
    (власній) точці обох."""
    d1 = orient(cx, cy, dx, dy, ax, ay)
    d2 = orient(cx, cy, dx, dy, bx, by)
    d3 = orient(ax, ay, bx, by, cx, cy)
    d4 = orient(ax, ay, bx, by, dx, dy)
    return d1 * d2 < 0 and d3 * d4 < 0

```

```

def point_in_polygon(px, py, poly):
    """Тест 'точка строго всередині простого полігона' методом кидання
    променя."""
    inside = False
    n = len(poly)
    j = n - 1
    for i in range(n):
        xi, yi = poly[i]
        xj, yj = poly[j]
        if (yi > py) != (yj > py):
            xint = (xj - xi) * (py - yi) / (yj - yi) + xi
            if px < xint:
                inside = not inside
        j = i
    return inside

```

```

def dist(ax, ay, bx, by):
    return math.hypot(ax - bx, ay - by)

```

```

#
=====
=====
# 2. Просторова сітка (рівномірна) для прискорення перевірок
видимості
#
=====
=====

```

```

class SpatialGrid:
    """Рівномірна сітка: ребра перешкод розкладаються по комітках, які

```

ВОНИ

перетинають. Для перевірки відрізка переглядаються тільки ребра у комірках уздовж нього (а не всі n ребер сцени)."""

```
def __init__(self, minx, miny, maxx, maxy, cell):
    self.minx = minx
    self.miny = miny
    self.cell = cell
    self.ncols = max(1, int((maxx - minx) / cell) + 1)
    self.nrows = max(1, int((maxy - miny) / cell) + 1)
    self.cells = {}          # (cx, cy) -> list[edge_id]
    self.edges = []         # edge_id -> (ax, ay, bx, by,
vid_a, vid_b)
    self._epoch = []        # позначки «вже перевірено»
(дедуплікація)
    self._cur = 0

def _cell_of(self, x, y):
    cx = int((x - self.minx) / self.cell)
    cy = int((y - self.miny) / self.cell)
    return cx, cy

def _cells_on_segment(self, ax, ay, bx, by):
    """Комірки, які перетинає відрізок ab (обхід комірок
Amanatides-Woo)."""
    cx, cy = self._cell_of(ax, ay)
    cx1, cy1 = self._cell_of(bx, by)
    out = [(cx, cy)]
    dx = bx - ax
    dy = by - ay
    stepx = 1 if dx > 0 else -1
    stepy = 1 if dy > 0 else -1

    if abs(dx) > EPS:
        nb = self.minx + (cx + (1 if dx > 0 else 0)) * self.cell
        t_max_x = (nb - ax) / dx
        t_delta_x = self.cell / abs(dx)
    else:
        t_max_x = INF
        t_delta_x = INF
    if abs(dy) > EPS:
        nb = self.miny + (cy + (1 if dy > 0 else 0)) * self.cell
        t_max_y = (nb - ay) / dy
        t_delta_y = self.cell / abs(dy)
    else:
        t_max_y = INF
        t_delta_y = INF
```

```

guard = self.ncols + self.nrows + 4
while (cx, cy) != (cx1, cy1) and guard > 0:
    if t_max_x < t_max_y:
        t_max_x += t_delta_x
        cx += stepx
    else:
        t_max_y += t_delta_y
        cy += stepy
    out.append((cx, cy))
    guard -= 1
return out

```

```

def add_edge(self, ax, ay, bx, by, vid_a, vid_b):
    eid = len(self.edges)
    self.edges.append((ax, ay, bx, by, vid_a, vid_b))
    for key in self._cells_on_segment(ax, ay, bx, by):
        self.cells.setdefault(key, []).append(eid)
    return eid

```

```

def seg_blocked(self, ax, ay, bx, by, skip_a, skip_b):
    """True, якщо відрізок ab власне перетинає якесь ребро
    перешкоди.

```

Комірки обходяться у порядку від a до b, тому перешкода виявляється рано і перевірка завершується достроково (ранній вихід).

Ребра,

```

    інцидентні до вершин skip_a/skip_b, ігноруються."""
    if len(self._epoch) != len(self.edges):
        self._epoch = [0] * len(self.edges)
    self._cur += 1
    ep = self._cur
    epoch = self._epoch
    edges = self.edges
    for key in self._cells_on_segment(ax, ay, bx, by):
        bucket = self.cells.get(key)
        if not bucket:
            continue
        for eid in bucket:
            if epoch[eid] == ep:
                continue
            epoch[eid] = ep
            ex0, ey0, ex1, ey1, ea, eb = edges[eid]
            if ea == skip_a or ea == skip_b or eb == skip_a or eb
== skip_b:
                continue
            if segments_properly_intersect(ax, ay, bx, by,
                                           ex0, ey0, ex1, ey1):
                return True

```

```
return False
```

```
#
=====
=====
# 3. Сцена: перешкоди, вершини, побудова графу видимості та пошук A*
#
=====
=====
```

```
class Vertex:
    __slots__ = ("x", "y", "poly", "idx", "size")

    def __init__(self, x, y, poly, idx, size):
        self.x = x          # координати
        self.y = y
        self.poly = poly    # id перешкоди
        self.idx = idx      # локальний індекс у полігоні
        self.size = size    # кількість вершин полігона
```

```
class Scene:
    """Геометрична сцена та алгоритм найкоротшого шляху.
```

```
    Відокремлена від GUI, тому її можна тестувати/заміряти без
    екрана."""
```

```
    def __init__(self):
        self.polygons = []    # list[list[(x, y)]]
        self.vertices = []    # list[Vertex] (глобальна нумерація
0..V-1)
        self.grid = None
        self.vis_cache = {}    # vid -> list[(vid, w)]  видимість
вершина↔вершина (точний режим)
        self.fast_cache = {}  # vid -> list[(vid, w)]  лише найближчі
(швидкий режим)
        self.vgrid = None     # сітка вершин (для пошуку найближчих
кандидатів)
        self.vcell = None
        self.S = None
        self.T = None
        self.S_ID = -1
        self.T_ID = -2
```

```
    # ---- побудова сцени
    -----
```

```

def clear(self):
    self.polygons.clear()
    self.vertices.clear()
    self.grid = None
    self.vis_cache.clear()
    self.fast_cache.clear()
    self.vgrid = None
    self.S = None
    self.T = None

def add_polygon(self, pts):
    """Додає перешкоду (список вершин у порядку обходу). Дублікати
    ігноруються."""
    if len(pts) < 3:
        return
    pid = len(self.polygons)
    self.polygons.append(list(pts))
    size = len(pts)
    for i, (x, y) in enumerate(pts):
        self.vertices.append(Vertex(x, y, pid, i, size))
    self.grid = None
    self.vis_cache.clear()
    self.fast_cache.clear()
    self.vgrid = None

def build_grid(self):
    """Передобробка: побудова просторової сітки з усіх ребер
    перешкод."""
    if not self.polygons:
        self.grid = SpatialGrid(0, 0, 1, 1, 1)
        return
    xs = [p[0] for poly in self.polygons for p in poly]
    ys = [p[1] for poly in self.polygons for p in poly]
    if self.S:
        xs.append(self.S[0]); ys.append(self.S[1])
    if self.T:
        xs.append(self.T[0]); ys.append(self.T[1])
    minx, maxx = min(xs) - 10, max(xs) + 10
    miny, maxy = min(ys) - 10, max(ys) + 10
    # розмір комірки ~ середня довжина ребра (баланс
    пам'ять/швидкість)
    total_edges = sum(len(p) for p in self.polygons)
    span = max(maxx - minx, maxy - miny)
    cell = max(8.0, span / max(8, int(math.sqrt(total_edges))) +
1))

    self.grid = SpatialGrid(minx, miny, maxx, maxy, cell)
    # глобальний індекс першої вершини кожного полігона
    base = 0

```

```

    for poly in self.polygons:
        m = len(poly)
        for i in range(m):
            a = poly[i]
            b = poly[(i + 1) % m]
            self.grid.add_edge(a[0], a[1], b[0], b[1], base + i,
base + (i + 1) % m)
            base += m
    # сітка вершин для пошуку найближчих кандидатів (швидкий
режим)
    self.vcell = cell * 1.5
    self.vgrid = {}
    for i, v in enumerate(self.vertices):
        key = (int((v.x - minx) / self.vcell), int((v.y - miny) /
self.vcell))
        self.vgrid.setdefault(key, []).append(i)
    self._vminx, self._vminy = minx, miny

```

---- ВИДИМІСТЬ

```

def _pos(self, vid):
    if vid == self.S_ID:
        return self.S
    if vid == self.T_ID:
        return self.T
    v = self.vertices[vid]
    return (v.x, v.y)

def _adjacent(self, i, j):
    """Чи є вершини i, j сусідніми у спільному полігоні (ребром
перешкоди)."""
    vi, vj = self.vertices[i], self.vertices[j]
    if vi.poly != vj.poly:
        return False
    d = abs(vi.idx - vj.idx)
    return d == 1 or d == vi.size - 1

def visible(self, uid, vid):
    """True, якщо відкритий відрізок між вершинами uid та vid не
перетинає жодну перешкоду (тобто вони взаємно видимі)."""
    ax, ay = self._pos(uid)
    bx, by = self._pos(vid)

    # випадок двох вершин одного полігона: хорда всередині
перешкоди => невидимі
    if uid >= 0 and vid >= 0:
        vu, vv = self.vertices[uid], self.vertices[vid]

```

```

        if vu.poly == vv.poly and not self._adjacent(uid, vid):
            mx, my = (ax + bx) / 2.0, (ay + by) / 2.0
            if point_in_polygon(mx, my, self.polygons[vu.poly]):
                return False

        # перетин з ребрами перешкод (ребра, інцидентні до uid/vid,
        # пропускаємо)
        return not self.grid.seg_blocked(ax, ay, bx, by, uid, vid)

    def vertex_neighbors(self, vid):
        """Список видимих вершин-перешкод (vid, вага) з кешуванням.
        Незалежний від S/T, тому обчислюється один раз і використовується у всіх
        запитах."""
        cached = self.vis_cache.get(vid)
        if cached is not None:
            return cached
        v = self.vertices[vid]
        res = []
        for j in range(len(self.vertices)):
            if j == vid:
                continue
            if self.visible(vid, j):
                w = self.vertices[j]
                res.append((j, dist(v.x, v.y, w.x, w.y)))
        self.vis_cache[vid] = res
        return res

    # ---- швидкий (наближений) режим: лише найближчі кандидати
    -----

    def _nearest_candidates(self, px, py, k):
        """~k найближчих вершин до (px, py) – кільцевий пошук по сітці
        вершин.
        Адаптивний: у щільних зонах кандидати близькі, у відкритих їх
        мало,
        тож захоплюються й далекі (тому шлях лишається майже
        оптимальним)."""
        cell = self.vcell
        cx = int((px - self._vminx) / cell)
        cy = int((py - self._vminy) / cell)
        found = []
        r = 0
        rmax = max(self.grid.ncols, self.grid.nrows) + 2
        while True:
            for gx in range(cx - r, cx + r + 1):
                for gy in range(cy - r, cy + r + 1):
                    if max(abs(gx - cx), abs(gy - cy)) != r:
                        continue
                    # лише «кільце» радіуса r

```

```

        bucket = self.vgrid.get((gx, gy))
        if bucket:
            found.extend(bucket)
        if (len(found) >= k and r >= 2) or r > rmax:
            break
        r += 1
    return found

def vertex_neighbors_fast(self, vid, k):
    """Видимі сусіди серед ~k найближчих вершин (з кешуванням).
    Швидкий режим:  $O(k)$  перевірок замість  $O(n)$  – для дуже щільних сцен
    1000+."""
    cached = self.fast_cache.get(vid)
    if cached is not None:
        return cached
    v = self.vertices[vid]
    res = []
    for j in self._nearest_candidates(v.x, v.y, k):
        if j == vid:
            continue
        if self.visible(vid, j):
            w = self.vertices[j]
            res.append((j, dist(v.x, v.y, w.x, w.y)))
    self.fast_cache[vid] = res
    return res

def build_full_visibility(self):
    """Повна передобробка: граф видимості для всіх вершин (для
    візуалізації та точних замірів). Складність  $O(n^2)$  – придатна лише для малих
    сцен."""
    if self.grid is None:
        self.build_grid()
    for i in range(len(self.vertices)):
        self.vertex_neighbors(i)

def build_knn_visibility(self, k=90):
    """k-найближчий граф видимості – той самий, що бачить ШВИДКИЙ
    режим A*. Складність  $O(n \cdot k)$  замість  $O(n^2)$ , тож будується у десятки разів
    швидше і дає набагато менше ребер. Використовується для візуалізації
    великих сцен, де повний граф був би і повільним, і нечитабельним
    «клубком»."""
    if self.grid is None:
        self.build_grid()
    for i in range(len(self.vertices)):

```



```

        self.vertex_neighbors_fast(i, k)

# ---- пошук найкоротшого шляху A*
-----

def _point_inside_any(self, p):
    for poly in self.polygons:
        if point_in_polygon(p[0], p[1], poly):
            return True
    return False

def shortest_path(self, S, T, fast=False, k=90):
    """Найкоротший шлях S->T методом A* з лінівним обчисленням
    видимості.

    fast=False – ТОЧНИЙ режим (гарантований оптимум).
    fast=True – ШВИДКИЙ режим: на кожному кроці розглядаються
    лише ~k
    найближчих вершин, тож для дуже щільних сцен
    (1000+ перешкод)
    час падає у десятки-сотні разів. Шлях завжди
    коректний (без
    перетину перешкод), але може бути на ~1% довшим
    за оптимум.

    Повертає словник зі шляхом, довжиною, часом і статистикою."""
    self.S, self.T = S, T
    if self.grid is None:
        self.build_grid()
    t0 = time.perf_counter()

    if self._point_inside_any(S) or self._point_inside_any(T):
        return {"path": None, "length": 0.0, "time": 0.0,
                "expanded": 0, "status": "S або T всередині
перешкоди"}

    Tx, Ty = T

    def h(vid):
        # евристика – пряма відстань
        x, y = self._pos(vid)
        return dist(x, y, Tx, Ty)

    def s_visible_vertices():
        if fast:
            cand = self._nearest_candidates(S[0], S[1], k)
        else:
            cand = range(len(self.vertices))

```

```

out = []
for j in cand:
    if self.visible(self.S_ID, j):
        w = self.vertices[j]
        out.append((j, dist(S[0], S[1], w.x, w.y)))
return out

def neighbors(uid):
    if uid == self.S_ID:
        out = s_visible_vertices()
        if self.visible(self.S_ID, self.T_ID):
            out.append((self.T_ID, dist(S[0], S[1], Tx, Ty)))
        return out
    if fast:
        out = list(self.vertex_neighbors_fast(uid, k))
    else:
        out = list(self.vertex_neighbors(uid))
    if self.visible(uid, self.T_ID):
        x, y = self._pos(uid)
        out.append((self.T_ID, dist(x, y, Tx, Ty)))
    return out

g = {self.S_ID: 0.0}
came = {}
closed = set()
openh = [(h(self.S_ID), 0.0, self.S_ID)]
expanded = 0

while openh:
    f, gu, u = heapq.heappop(openh)
    if u in closed:
        continue
    closed.add(u)
    expanded += 1
    if u == self.T_ID:
        break
    for v, w in neighbors(u):
        ng = gu + w
        if ng < g.get(v, INF) - EPS:
            g[v] = ng
            came[v] = u
            heapq.heappush(openh, (ng + h(v), ng, v))

elapsed = time.perf_counter() - t0
if self.T_ID not in came and self.T_ID not in closed:
    if fast:
        # у швидкому режимі обмеження кандидатів (~k
найближчих) могло
        # розірвати граф і дати хибне «шляху немає» – відкат

```

ДО ТОЧНОГО

```
        return self.shortest_path(S, T, fast=False)
    return {"path": None, "length": 0.0, "time": elapsed,
            "expanded": expanded, "status": "Шлях не існує"}
```

```
    # відновлення шляху
    path_ids = [self.T_ID]
    cur = self.T_ID
    while cur != self.S_ID:
        cur = came[cur]
        path_ids.append(cur)
    path_ids.reverse()
    path = [self._pos(vid) for vid in path_ids]
    length = g[self.T_ID]
    return {"path": path, "length": length, "time": elapsed,
            "expanded": expanded, "status": "OK",
            "mode": "швидкий" if fast else "точний"}
```

#

```
=====
=====
```

4. Генератор перешкод (для авто-режиму та замірів ефективності)

#

```
=====
=====
```

```
def random_convex_polygon(cx, cy, r, k, rng):
    """Випадковий опуклий полігон із k вершин навколо центру (cx,
    cy)."""
    angles = sorted(rng.uniform(0, 2 * math.pi) for _ in range(k))
    pts = []
    for a in angles:
        rad = r * rng.uniform(0.6, 1.0)
        pts.append((cx + rad * math.cos(a), cy + rad * math.sin(a)))
    return pts
```

```
def generate_scene(scene, n_obstacles, width, height, rng,
                   vmin=3, vmax=6, rmin=7, rmax=17):
    """Заповнює сцену n_obstacles випадковими опуклими перешкодами."""
    scene.clear()
    for _ in range(n_obstacles):
        cx = rng.uniform(rmax, width - rmax)
        cy = rng.uniform(rmax, height - rmax)
        r = rng.uniform(rmin, rmax)
        k = rng.randint(vmin, vmax)
        scene.add_polygon(random_convex_polygon(cx, cy, r, k, rng))
```

```

# S, T – у вільних точках
def free_point():
    for _ in range(2000):
        p = (rng.uniform(5, width - 5), rng.uniform(5, height -
5))
        if not scene._point_inside_any(p):
            return p
    return (5, 5)
scene.S = free_point()
scene.T = free_point()
scene.build_grid()

def generate_worst_case(scene, n_walls, width, height):
    """Найгірший ввід – «слалом». Кожна колонка-стіна сягає обох країв
(верхнього
й нижнього) і має єдиний прохід, чия висота почергово то вгорі, то
внизу.
Тому пройти крізь сцену неможливо інакше, ніж зигзагом крізь усі
проходи:
шлях має  $\theta(n\_walls)$  ланок, а A* розкриває усі вершини – це й
демонструє
найгіршу (квадратичну) поведінку алгоритму."""
    scene.clear()
    ft = 60.0 # товщина рамки
    rect = lambda x0, y0, x1, y1: scene.add_polygon(
        [(x0, y0), (x1, y0), (x1, y1), (x0, y1)])
    # замкнена рамка: S і T опиняються «в пастці», шлях не може її
обійти
    rect(0, 0, width, ft) # верх
    rect(0, height - ft, width, height) # низ
    rect(0, 0, ft, height) # ліво
    rect(width - ft, 0, width, height) # право

    yc0, yc1 = ft, height - ft # межі коридору
    corr = yc1 - yc0
    upper = yc0 + 0.30 * corr # центр верхнього
проходу
    lower = yc0 + 0.70 * corr # центр нижнього
проходу
    gap_half = 0.12 * corr
    step = (width - 2 * ft) / (n_walls + 1)
    # ширина стіни – завжди частка кроку, тож між колонами лишається
проміжок
    # (інакше при великій к-сті стін колони злипаються й коридор
замуровується)
    wall_w = step * 0.4
    for i in range(n_walls):
        x = ft + step * (i + 1)

```

```

        c = upper if i % 2 == 0 else lower
        xL, xR = x - wall_w / 2, x + wall_w / 2
        # колона з двох частин (перекриває рамку, аби не було щілин
для ковзання)
        rect(xL, ft - 10, xR, c - gap_half)          # верхня
частина
        rect(xL, c + gap_half, xR, height - ft + 10) # нижня частина
        scene.S = (ft + step * 0.5, upper)
        scene.T = (width - ft - step * 0.5,
                    upper if (n_walls - 1) % 2 == 0 else lower)
        scene.build_grid()

```

```

#
=====
=====
# 5. Графічний інтерфейс (Tkinter)
#
=====
=====

```

```

def run_gui():
    import tkinter as tk
    from tkinter import ttk

    W, H = 980, 720

    scene = Scene()
    rng = random.Random(12345)

    state = {
        "mode": "draw",          # draw | set_s | set_t
        "current": [],           # вершини полігона, що малюється
        "show_graph": False,
        "graph_knn": False,      # True → малюємо k-найближчий граф
(великі сцени)
        "last_result": None,
    }

    root = tk.Tk()
    root.title("Найкоротші шляхи на множині перешкод у 2D")

    left = ttk.Frame(root, padding=8)
    left.pack(side="left", fill="y")
    canvas = tk.Canvas(root, width=W, height=H, bg="white",
highlightthickness=1,
                        highlightbackground="#888")
    canvas.pack(side="right", fill="both", expand=True)

```

```

# ---- елементи керування
-----
    ttk.Label(left, text="Режим вводу", font=("TkDefaultFont", 10,
"bold")).pack(anchor="w")
    mode_var = tk.StringVar(value="draw")

    def set_mode():
        state["mode"] = mode_var.get()
        status(f"Режим: {mode_var.get()}")

    ttk.Radiobutton(left, text="Малювати перешкоду",
variable=mode_var,
                    value="draw", command=set_mode).pack(anchor="w")
    ttk.Radiobutton(left, text="Поставити S (старт)",
variable=mode_var,
                    value="set_s", command=set_mode).pack(anchor="w")
    ttk.Radiobutton(left, text="Поставити T (фініш)",
variable=mode_var,
                    value="set_t", command=set_mode).pack(anchor="w")

    ttk.Separator(left, orient="horizontal").pack(fill="x", pady=6)
    ttk.Label(left, text="Ручний ввід (мишкою):").pack(anchor="w")
    ttk.Label(left, text="ЛКМ – додати вершину\nПКМ / кнопка –
завершити перешкоду",
                    foreground="#555").pack(anchor="w")
    ttk.Button(left, text="Завершити перешкоду",
                    command=lambda: finish_polygon()).pack(fill="x",
pady=2)

    ttk.Separator(left, orient="horizontal").pack(fill="x", pady=6)
    ttk.Label(left, text="Авто-генерація", font=("TkDefaultFont", 10,
"bold")).pack(anchor="w")
    ttk.Label(left, text="К-сть перешкод:").pack(anchor="w")
    n_var = tk.StringVar(value="300")
    ttk.Entry(left, textvariable=n_var, width=10).pack(anchor="w")
    ttk.Button(left, text="Згенерувати (випадково)",
                    command=lambda: do_generate()).pack(fill="x", pady=2)
    ttk.Button(left, text="Найгірший випадок (слалом)",
                    command=lambda: do_worst()).pack(fill="x", pady=2)

    ttk.Separator(left, orient="horizontal").pack(fill="x", pady=6)
    ttk.Button(left, text="► Знайти найкоротший шлях",
                    command=lambda: do_run()).pack(fill="x", pady=2)
    ttk.Label(left, text="(швидкий режим вмикається\nавтоматично при
>1000 вершин)",
                    foreground="#777").pack(anchor="w")
    show_var = tk.BooleanVar(value=False)

```

```

ttk.Checkbutton(left, text="Показати граф видимості",
                variable=show_var,
                command=lambda: toggle_graph()).pack(anchor="w")
ttk.Button(left, text="Очистити робочу область",
           command=lambda: do_clear()).pack(fill="x", pady=2)

ttk.Separator(left, orient="horizontal").pack(fill="x", pady=6)
info = tk.Text(left, width=30, height=12, font=("TkFixedFont", 9),
               relief="flat", bg="#f4f4f4")
info.pack(fill="x")

status_var = tk.StringVar(value="Готово.")
ttk.Label(left, textvariable=status_var, foreground="#06c",
          wraplength=210).pack(anchor="w", pady=(6, 0))

def status(msg):
    status_var.set(msg)

def set_info(lines):
    info.delete("1.0", "end")
    info.insert("1.0", "\n".join(lines))

# ---- малювання
-----
def redraw():
    canvas.delete("all")
    # перешкоди
    for poly in scene.polygons:
        flat = [c for p in poly for c in p]
        if len(flat) >= 6:
            canvas.create_polygon(*flat, fill="#c9ccd3",
outline="#5b5f66")
            # граф видимості: повний (малі сцени) або k-найближчий
(великі)
            graph_src = scene.fast_cache if state["graph_knn"] else
scene.vis_cache
            if state["show_graph"] and graph_src:
                for i, lst in graph_src.items():
                    ax, ay = scene._pos(i)
                    for j, _ in lst:
                        if j > i:
                            bx, by = scene._pos(j)
                            canvas.create_line(ax, ay, bx, by,
fill="#e7c7c7")
                            # поточний полігон, що малюється
                            cur = state["current"]
                            for (x, y) in cur:
                                canvas.create_oval(x - 3, y - 3, x + 3, y + 3,
fill="#333")

```

```

        for i in range(len(cur) - 1):
            canvas.create_line(*cur[i], *cur[i + 1], fill="#333",
dash=(3, 2))
        # шлях
        res = state["last_result"]
        if res and res.get("path"):
            pts = res["path"]
            flat = [c for p in pts for c in p]
            canvas.create_line(*flat, fill="#1565ff", width=3)
        # S, T
        if scene.S:
            x, y = scene.S
            canvas.create_oval(x - 6, y - 6, x + 6, y + 6,
fill="#16a34a", outline="")
            canvas.create_text(x, y - 12, text="S", fill="#16a34a",
font=("TkDefaultFont", 11, "bold"))

        if scene.T:
            x, y = scene.T
            canvas.create_oval(x - 6, y - 6, x + 6, y + 6,
fill="#dc2626", outline="")
            canvas.create_text(x, y - 12, text="T", fill="#dc2626",
font=("TkDefaultFont", 11, "bold"))

    def refresh_info():
        nv = len(scene.vertices)
        nh = len(scene.polygons)
        lines = [f"Перешкод (h):    {nh}",
f"Вершин (n):    {nv}"]
        res = state["last_result"]
        if res:
            lines.append("")
            lines.append(f"Статус: {res['status']}")
            if res.get("mode"):
                lines.append(f"Режим: {res['mode']}")
            if res.get("path"):
                lines.append(f"Довжина шляху: {res['length']:.1f}")
                lines.append(f"Ланок у шляху: {len(res['path'])} -
1}")
            lines.append(f"Вершин розкрито: {res['expanded']}")
            lines.append(f"Час запиту: {res['time'] * 1000:.1f}
мс")
        set_info(lines)

    # ---- обробники подій
    -----
    def on_left(ev):
        x, y = ev.x, ev.y
        m = state["mode"]
        if m == "draw":
            state["current"].append((x, y))

```



```

    elif m == "set_s":
        scene.S = (x, y)
        scene.grid = None
        status("S встановлено")
    elif m == "set_t":
        scene.T = (x, y)
        scene.grid = None
        status("T встановлено")
    redraw()

def finish_polygon():
    cur = state["current"]
    if len(cur) >= 3:
        scene.add_polygon(cur)
        status(f"Додано перешкоду з {len(cur)} вершин")
    else:
        status("Потрібно щонайменше 3 вершини")
    state["current"] = []
    state["last_result"] = None
    redraw()
    refresh_info()

def on_right(ev):
    finish_polygon()

def do_generate():
    try:
        n = int(n_var.get())
    except ValueError:
        status("Некоректне число")
        return
    n = max(1, min(n, 6000))
    t0 = time.perf_counter()
    generate_scene(scene, n, canvas.winfo_width() or W,
                  canvas.winfo_height() or H, rng)
    dt = time.perf_counter() - t0
    state["current"] = []
    state["last_result"] = None
    status(f"Згенеровано {n} перешкод ({len(scene.vertices)}
вершин) за {dt*1000:.0f} мс")
    redraw()
    refresh_info()

def do_worst():
    try:
        nw = int(n_var.get())
    except ValueError:
        status("Некоректне число")
        return

```

```

nw = max(2, min(nw, 400))
generate_worst_case(scene, nw, canvas.winfo_width() or W,
                    canvas.winfo_height() or H)
state["current"] = []
state["last_result"] = None
status(f"Слалом: {nw} стін ({len(scene.vertices)} вершин) –
найгірший ввід")
redraw()
refresh_info()

def do_run():
    if scene.S is None or scene.T is None:
        status("Спочатку задайте S і T")
        return
    if scene.grid is None:
        scene.build_grid()
    # швидкий режим вмикається автоматично для великих сцен (>1000
вершин)
    auto_fast = len(scene.vertices) > 1000
    res = scene.shortest_path(scene.S, scene.T, fast=auto_fast)
    state["last_result"] = res
    mode = res.get("mode", "")
    status(f"{res['status']} ({mode}) | {res['time']*1000:.1f} мс
| розкрито {res['expanded']}")
    redraw()
    refresh_info()

# пороги візуалізації графу видимості
GRAPH_FULL_MAX = 400      # ≤ – повний граф  $O(n^2)$  (гарантований,
але важкий)
GRAPH_KNN_MAX = 3000      # ≤ – k-найближчий граф  $O(n \cdot k)$  (швидкий,
читабельний)

def toggle_graph():
    state["show_graph"] = show_var.get()
    if not state["show_graph"]:
        redraw()
        return
    n = len(scene.vertices)
    if scene.grid is None:
        scene.build_grid()
    if n <= GRAPH_FULL_MAX:
        # малі сцени – повний граф видимості (точний)
        state["graph_knn"] = False
        scene.build_full_visibility()
        status(f"Повний граф видимості ({n} вершин)")
    elif n <= GRAPH_KNN_MAX:
        # великі сцени – k-найближчий граф (той, що бачить швидкий
режим A*)

```

```

        state["graph_knn"] = True
        t0 = time.perf_counter()
        scene.build_knn_visibility(90)
        dt = (time.perf_counter() - t0) * 1000
        edges = sum(len(l) for l in scene.fast_cache.values()) //
2
        status(f"k-найближчий граф ({n} вершин, {edges} ребер) за
{dt:.0f} мс")
        else:
            # надто щільно навіть для k-NN – малювати марно
            (нечитабельний клубок)
            status(f"Граф видимості вимкнено: {n} вершин >
{GRAPH_KNN_MAX} "
                f"(був би нечитабельний клубок)")
            state["show_graph"] = False
            show_var.set(False)
            redraw()

def do_clear():
    scene.clear()
    state["current"] = []
    state["last_result"] = None
    status("Очищено")
    redraw()
    refresh_info()

canvas.bind("<Button-1>", on_left)
canvas.bind("<Button-3>", on_right)          # ПКМ
canvas.bind("<Button-2>", on_right)          # середня кнопка /
трекпад
root.bind("<Return>", lambda e: do_run())
root.bind("<Escape>", lambda e: do_clear())

refresh_info()
redraw()
root.mainloop()

#
=====
=====
# 6. Перевірка коректності та заміри ефективності (без GUI)
#
=====
=====

def selftest():
    print("=== Перевірка коректності ===")

```

```

sc = Scene()
# один квадрат-перешкода у центрі; S зліва, T справа
sc.add_polygon([(40, 40), (60, 40), (60, 60), (40, 60)])
sc.build_grid()
res = sc.shortest_path((10, 50), (90, 50))
assert res["status"] == "OK", res
straight = 80.0
print(f" Квадрат: довжина={res['length']:.3f} (пряма={straight}),
шлях обходить перешкоду: "
      f"{res['length'] > straight}")
assert res["length"] > straight, "шлях має бути довшим за пряму
крізь перешкоду"
# точки на одній лінії без перешкод
sc2 = Scene(); sc2.build_grid()
r2 = sc2.shortest_path((0, 0), (100, 0))
print(f" Без перешкод: довжина={r2['length']:.3f} (очікувано
100.0)")
assert abs(r2["length"] - 100.0) < 1e-6
# S всередині перешкоди
sc3 = Scene()
sc3.add_polygon([(0, 0), (100, 0), (100, 100), (0, 100)])
sc3.build_grid()
r3 = sc3.shortest_path((50, 50), (200, 200))
print(f" S всередині: статус='{r3['status']}'")
assert r3["path"] is None
print(" Усі перевірки пройдено ✓")

```

```

def bench(n_obstacles):
    print(f"=== Заміри ефективності: {n_obstacles} перешкод ===")
    sc = Scene()
    rng = random.Random(7)
    # світ зростає  $\propto \sqrt{h}$  – заміри при сталій густині перешкод
    side = int(70 * math.sqrt(n_obstacles))
    generate_scene(sc, n_obstacles, side, side, rng)
    n = len(sc.vertices)
    t0 = time.perf_counter()
    sc.build_grid()
    t_grid = time.perf_counter() - t0
    # перший запит (наповнює кеш видимості – роль передобробки)
    res = sc.shortest_path(sc.S, sc.T)
    # серія повторних запитів для різних S, T (модель задачі)
    times = []
    for _ in range(8):
        S = sc.S
        for _ in range(200):
            p = (rng.uniform(0, side), rng.uniform(0, side))
            if not sc._point_inside_any(p):
                T = p
                break

```

```

        r = sc.shortest_path(S, T)
        times.append(r["time"])
    times.sort()
    median = times[len(times) // 2]
    print(f"  h={n_obstacles} перешкод, n={n} вершин")
    print(f"  Сітка (передобробка): {t_grid*1000:8.1f} мс")
    print(f"  1-й запит (з прогрівом): {res['time']*1000:8.1f} мс
(розкрито {res['expanded']})")
    print(f"  повторні запити, медіана:{median*1000:8.1f} мс "
          f"(діапазон {times[0]*1000:.0f}..{times[-1]*1000:.0f} мс)")
    if res.get("path"):
        print(f"  Довжина шляху: {res['length']:.1f}, ланок
{len(res['path'])-1}")

```

```

def bench_worst(n_walls):
    print(f"=== Найгірший випадок (слалом): {n_walls} стін ===")
    sc = Scene()
    side = max(800, n_walls * 24)
    generate_worst_case(sc, n_walls, side, side)
    n = len(sc.vertices)
    res = sc.shortest_path(sc.S, sc.T)
    print(f"  стін={n_walls}, n={n} вершин")
    print(f"  запит: {res['time']*1000:8.1f} мс (розкрито
{res['expanded']}, "
          f"ланок {len(res['path'])-1 if res.get('path') else 0},
статус {res['status']})")
    if res.get("path"):
        print(f"  довжина шляху: {res['length']:.1f}")

```

```

def bench_fast(n_obstacles):
    """Порівняння точного і швидкого режимів на щільній сцені
(фіксоване полотно)."""
    print(f"=== Точний vs Швидкий режим: {n_obstacles} перешкод ===")
    sc = Scene()
    rng = random.Random(0)
    generate_scene(sc, n_obstacles, 980, 720, rng)
    n = len(sc.vertices)
    sc.vis_cache.clear(); sc.fast_cache.clear()
    ex = sc.shortest_path(sc.S, sc.T, fast=False)
    fa = sc.shortest_path(sc.S, sc.T, fast=True)
    print(f"  h={n_obstacles}, n={n} вершин")
    print(f"  точний: {ex['length']:.1f} за {ex['time']*1000:8.1f}
мс")
    print(f"  швидкий: {fa['length']:.1f} за {fa['time']*1000:8.1f}
мс")
    if ex.get("path") and fa.get("path"):
        err = (fa['length'] - ex['length']) / ex['length'] * 100

```

```

        print(f"   похибка швидкого: {err:+.3f}% | прискорення: "
              f"{ex['time']/max(fa['time'],1e-9):.0f}x")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--selftest":
        selftest()
    elif len(sys.argv) > 1 and sys.argv[1] == "--bench":
        n = int(sys.argv[2]) if len(sys.argv) > 2 else 1000
        bench(n)
    elif len(sys.argv) > 1 and sys.argv[1] == "--bench-worst":
        n = int(sys.argv[2]) if len(sys.argv) > 2 else 30
        bench_worst(n)
    elif len(sys.argv) > 1 and sys.argv[1] == "--bench-fast":
        n = int(sys.argv[2]) if len(sys.argv) > 2 else 1500
        bench_fast(n)
    else:
        run_gui()

```